

COMPUTE-PREFIX-FUNCTION (KMP)

Understanding the Construction of
the Pi Array
Algorithm Analysis & Trace

Algorithm

- **COMPUTE- PREFIX- FUNCTION (P)**

1. $m \leftarrow \text{length } [P] // \text{'p' pattern to be matched}$
2. $\Pi[1] \leftarrow 0$
3. $k \leftarrow 0$
4. for $q \leftarrow 2$ to m
5. do while $k > 0$ and $P[k + 1] \neq P[q]$
6. do $k \leftarrow \Pi[k]$
7. If $P[k + 1] = P[q]$
8. then $k \leftarrow k + 1$
9. $\Pi[q] \leftarrow k$
10. Return Π

What is the Pi (Π) Array?

- The Goal: Build a map to backtrack efficiently on mismatches.
- Definition: $\Pi[q]$ stores the length of the Longest Proper Prefix of $P[1..q]$ that is also a Suffix of $P[1..q]$.
- Why? It prevents re-scanning characters we have already matched.

Understanding the Variables

- P: The Pattern string (length m).
- q : The current character index in the pattern (loop iterator).
- k : The length of the matching prefix found so far.
 - Think of ' k ' as the candidate length for the next match.

How the Algorithm Moves

1. Initialization: $\Pi[1] = 0, k = 0$.
2. Iterate (q from 2 to m).
3. Mismatch Loop (while):
 - If $P[k+1] \neq P[q]$, we backtrack: $k \leftarrow \Pi[k]$.
4. The Match (if):
 - If $P[k+1] == P[q]$, extend prefix: $k \leftarrow k + 1$.
5. Record: Store k in $\Pi[q]$.

Trace Example: P = 'ababaca'

- Pattern: a b a b a c a
- Index: 1 2 3 4 5 6 7
- Initialization:
- $m = 7$
- $\Pi[1] = 0$
- $k = 0$ (current match length)

Iterations $q=2$ and $q=3$

Iteration $q=2$ (Char 'b'):

- Compare $P[k+1]$ ('a') vs $P[q]$ ('b') $\rightarrow$ Mismatch.
- k stays 0.
- $\Pi[2] \leftarrow 0$

Iteration $q=3$ (Char 'a'):

- Compare $P[k+1]$ ('a') vs $P[q]$ ('a') $\rightarrow$ Match!
- Increment $k \rightarrow 1$.
- $\Pi[3] \leftarrow 1$

Iterations $q=4$ and $q=5$ (Building Prefix)

Iteration $q=4$ (Char 'b'):

- Current $k=1$. Match $P[2]$ ('b') vs $P[4]$ ('b').
- $k \rightarrow 2$.
- $\Pi[4] \leftarrow 2$

Iteration $q=5$ (Char 'a'):

- Current $k=2$. Match $P[3]$ ('a') vs $P[5]$ ('a').
- $k \rightarrow 3$.
- $\Pi[5] \leftarrow 3$ (Matches 'aba')

Iteration $q=6$ (The Critical Backtrack)

Current State: $k=3$, processing $P[6]='c'$. Expected $P[4]='b'$.

1. Compare 'b' vs 'c' $\rightarrow$ Mismatch.
2. WHILE loop ($k>0$): Backtrack $k \leftarrow \Pi[3]$ (which is 1).
3. Compare $P[2]$ ('b') vs 'c' $\rightarrow$ Mismatch.
4. WHILE loop ($k>0$): Backtrack $k \leftarrow \Pi[1]$ (which is 0).
5. Stop. $\Pi[6] \leftarrow 0$.

Iteration q=7 (Final Step)

Current State: $k=0$, processing $P[7]='a'$.

1. Compare $P[1]('a')$ vs $P[7]('a')$ $\rightarrow$ Match.
2. Increment $k \rightarrow 1$.
3. $\Pi[7] \leftarrow 1$.

Final Array: $[0, 0, 1, 2, 3, 0, 1]$

Complexity Analysis

Time Complexity: $O(m)$

- Variable 'k' increments at most m times.
- Variable 'k' cannot decrease more times than it increased.
- Amortized analysis proves linear time.

Space Complexity: $O(m)$ to store the Π array.